

9.4 Open WebUI for Ollama

Regardless of whether you want to use Ollama for general prompts or for use in development, the command line is probably not the most convenient way. For this reason, there are a number of options for equipping the platform with an appealing and user-friendly frontend. One of the most popular ones is *Open WebUI*, formerly known as Ollama WebUI.

As the name change suggests, Open WebUI was originally developed as a frontend for Ollama. However, the project supports not only Ollama but also any LLM runner with an OpenAI-compatible API. Open WebUI provides a number of functions that make it interesting both for local operation and for installation in companies:

- **Easy installation**

The core idea is that you host Open WebUI yourself. Like Ollama, Open WebUI also provides numerous installation options. Probably the simplest and cleanest method is to use the official Docker image of the project. But we'll return to the installation later.

- **Modern UI**

The UI is very similar to the ChatGPT web interface and is delivered as a responsive progressive web app (PWA). This means that you can use the website on any device, and it adapts to different resolutions. You can also install the website on your device so that you no longer notice any difference between a native app and the website.

- **Creation of models**

Open WebUI provides you with a graphical interface that enables you to generate user-defined models for Ollama.

- **Local RAG integration**

Similar to GPT4All, Open WebUI allows you to integrate your own documents into your prompts via RAG.

- **Web search for RAG**

In addition to the RAG feature with local documents, you can also give

your local AI access to the internet and enrich the results of the queries with the search results from the internet.

- **Image generation**

Open WebUI supports the integration of image generators such as ComfyUI or DALL-E.

- **Cross-model chats**

You can combine several models in one prompt at the same time and use their individual strengths.

- **Role-based access control**

One feature that makes Open WebUI particularly interesting in a larger context is the support of role-based access control (RBAC). This feature allows you to grant certain people access to the UI and exclude others.

9.4.1 Installing Open WebUI

In the documentation, Open WebUI recommends installing it as a container. The prerequisite is that you've installed a container platform on your system. There's a Docker image for Open WebUI that you can start as a container via the following command:

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:main
```

Once the container has been started, you can access the Open WebUI web interface via *http://localhost:3000*. The user login feature is activated by default. If you want to use the tool in single-user mode and do without a login, you can deactivate this feature using the `WEBUI_AUTH` environment variable with the `False` value. If you use the standard mode, you'll first see a login screen when you open the UI for the first time. You can create an initial

user account via the **Sign up** link, which is automatically activated and has admin privileges.

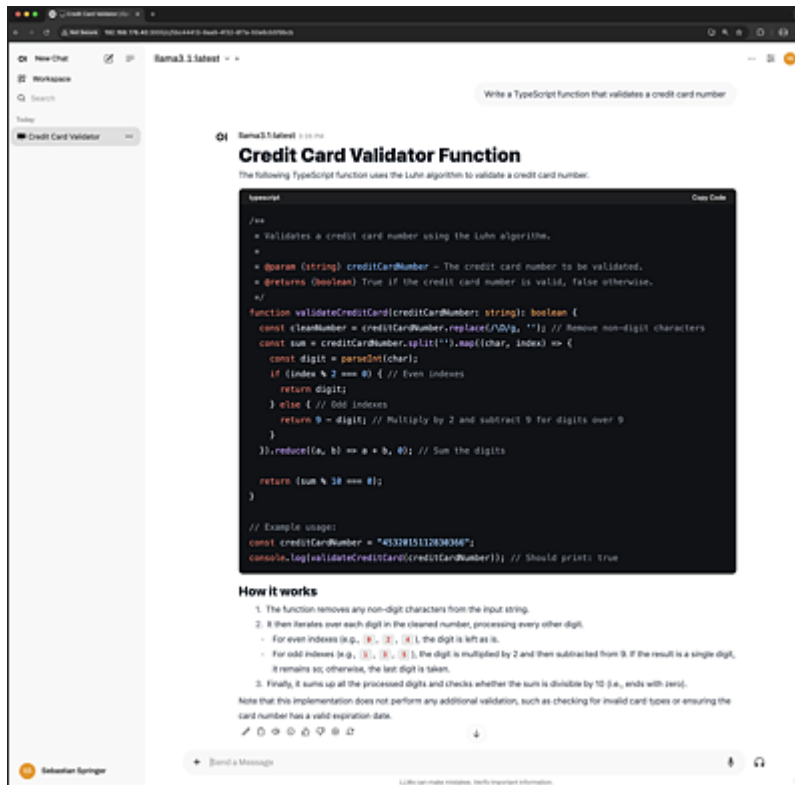


Figure 9.2 Web Interface of Open WebUI

Before you start prompting, you must make sure that Ollama is started and accessible as a backend. If you run both applications in separate Docker containers, communication usually works without any further adjustments. You can then select one of the models provided by Ollama via the **Select a model** dropdown menu. By clicking on **Set as default**, you set this model as the default for all future sessions and don't have to select it again each time.

With the combination of Ollama and Open WebUI, you're independent of AI providers such as OpenAI, Google, or Anthropic. Your data remains on your system, and you can control the performance of your system depending on how many local resources you invest. Models such as the 405B variant of Llama 3.1 have no reason to hide behind the large commercial providers, but they require an extremely large amount of storage space, RAM, and computing power.

Thanks to the GUI of Open WebUI, operating the services is very convenient. In the following sections, you'll learn more about the possibilities

provided by the combination of Ollama and Open WebUI.

9.4.2 Using Custom Documents in Open WebUI

If you use LLMs for development, the results of prompts on general questions are particularly impressive. However, the more specific the questions get, the greater the probability that the AI tool will be wrong. In the best-case scenario, it admits this and tells you that it has reached the end of its knowledge. However, it's more likely that you'll simply receive the wrong answer.

By using RAG, however, you can help the AI tool and convey specific specialist knowledge to it. You've already become acquainted with a variant of this with the LocalDocs of GPT4All, and [Chapter 12](#) of this book deals with this topic in even greater detail. Open WebUI also provides the option of uploading your own documents and enriching the answers to your prompts with them. For example, you can upload manuals and documentation for frameworks, libraries, or programming languages; prompt specific issues; and receive a suitable answer.

To use RAG in Open WebUI, you must first upload one or more documents. To do this, navigate to **Workspace**, and select the **Documents** tab. Initially, you'll see an empty list here, as no documents are available yet. Documents can be uploaded by using the plus button. Open WebUI uses tags for documents and collections. Each document is given an individual name tag that you can use to reference it. You can also assign any number of tags to each document. Documents with identical tags are combined into collections. As soon as you've uploaded at least one document, you can enter the hash (#) as the first character in a prompt. Open WebUI then gives you a selection of the available tags whose documents you can integrate into the current session via a dropdown menu.

The RAG feature of Open WebUI supports a variety of file formats, including text files such as source code files, Markdown, XML, Microsoft Excel, PDF, or EPUB. Internally, Open WebUI processes these files via various loaders.

You can configure the RAG feature under **Documents** in the admin panel, which you can access via the admin user settings. There you specify which embedding model engine you want to use. The default engine is **SentenceTransformers**. This is a Python library that converts sentences and text fragments into vectors (embeddings). The RAG is then based on these vectors.

The RAG functionality of Open WebUI isn't limited to local documents. Instead of a tag that references a document, you can also enter a URL to a website in the prompt after the hash (#) and select the suggested link. The application then reads the page and uses its structure to generate a response. An example of such a web-based prompt could read as follows:

Prompt: #https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray

Is it possible to freeze a TypedArray?

No, it is not possible to freeze a TypedArray. According to the rules, TypedArrays that aren't empty cannot be frozen.

The answer will provide you with additional information about where the information was found in the source.

If you reference web pages, you should make sure that they contain as much structured text as possible because Open WebUI processes and uses this text for the answers. Ideally, the website should provide a separate reading view, as this is best suited to the web-based RAG.

The quality of the results depends heavily on the model used. If you use a small model with a small number of parameters, such as the 8B variant of Llama 3.1, you'll obtain significantly poorer quality results than with the 405B variant of the model.

The ability to search in documents and on websites is particularly helpful when you look up documentation, and it can speed up your development process considerably. The risk of receiving incorrect answers is also lower,

as Open WebUI also provides references, and you can check the relevant text passage in the documentation.

9.4.3 Custom Workspace Models

The LLMs play a prominent role in both Ollama and Open WebUI; without them, both tools are virtually useless. It's therefore of great importance that the models are managed properly. Open WebUI makes it easier to use the models by providing a GUI through which you can perform various operations. You can access model management via the navigation in the left-hand column of the interface and the **Workspace** item. The first tab of the workspace is dedicated to models. There, you can see the list of models installed in the connected Ollama by default. You can clone, edit, or delete an existing model. You can also create a new model or download one of the community models.

If you want to integrate an entirely new model, this doesn't work via the workspace. Instead, you need to go to the admin panel, select the **Settings** tab and then **Models**. You'll then be able to download one of the official models from the Ollama library via **Pull a model from Ollama.com**. For example, you can enter `phi3:medium`, and Open WebUI will cause Ollama to download the Phi3 model in its 14B variant. Once this process is complete, you can use the model in both Ollama and Open WebUI.

Another way to integrate a new model is via the *Grokking Generative Universal Format* (GGUF) file of a model, which you can download. GGUF is a file format for storing and exchanging LLMs. Currently, the import of GGUF files is still hidden behind the **Experimental** item in the **Model** section of the admin panel. If you have the GGUF file of a model, for example, from Gemma2, locally on your system, you can upload this file using the **Upload** button and integrate the model into Ollama. This variant is particularly interesting for models that aren't part of the Ollama library. A list of ready-made GGUF files you can use can be found on Hugging Face at <https://huggingface.co/models?search=gguf>.

No matter how you integrate a model into Ollama, Open WebUI allows you to customize it for your purposes. To do this, you either edit an existing

model in the workspace or create a new model there based on an existing Ollama model. You can use the following fields:

- **Name and model ID**

These two fields identify your model.

- **Base Model (From)**

This is where you can select the underlying model. All models installed on your local Ollama are available for selection in this dropdown menu.

- **Description**

Here, you can enter a description that describes your user-defined model in more detail.

- **System prompt**

You can use the system prompt to store statements that aren't visible when interacting via the chat interface, but which influence the behavior of the model. Examples of such system prompts include the following:

- Use formal language.
- Ignore certain requests.
- Prioritize technical accuracy.

- **Advanced Params**

This section gives you very fine control over various aspects of the model. Here, for example, you can configure the sampling temperature and thus specify whether the output should be more random or more focused, or set the size of the context.

- **Prompt Suggestions**

You can use the prompt suggestions to store one or more suggestions for prompts. An example of such a prompt suggestion is "How do I implement a debounce function in JavaScript?" These suggestions get displayed when you start a new session with your model.

- **Knowledge, Tools, Filters, Actions**

These items refer to Open WebUI-specific extensions for your model:

- **Knowledge** stands for local documents that you want to make available to your model by default. This allows you to create a model that specializes in clean code and architecture. In this case, you can integrate suitable documents.
- **Tools** stand for various aids for your chats, such as web search or API interactions.

- **Filters** can be used to apply actions to the messages of your users or the model. For example, you can prevent certain messages, automatically translate messages, or limit the number of messages.
- **Actions** stand for buttons in the message bar for your end users. These buttons allow your users to perform certain actions such as formatting code.
- **Capabilities**
You can select the **Vision** option here, which you can activate or deactivate. The option stands for the model's ability to process and interpret visual information.
- **Tags**
You can use the tags to add compact meta information for your model.

You can export the configuration of your model as a JavaScript Object Notation (JSON) structure to create a backup or share the preconfigured model with other people. The following code is an example of such a model export:

```
[
  {
    "id": "code-assistant",
    "name": "Code Assistant",
    "object": "model",
    "created": 1723984665,
    "owned_by": "ollama",
    "info": {
      "id": "code-assistant",
      "user_id": "befb1b0f-171a-41d8-b955-5283e2e45582",
      "base_model_id": "llama3.1:latest",
      "name": "Code Assistant",
      "params": {
        "system": "You are a Code Assistant, a developer's expert tool. Provide clear, best-practice code, technical advice, and debugging help for languages like JavaScript, TypeScript, and Python. Keep responses concise and"
```



```

        implementation-ready.",
        "num_ctx": 4096
    },
    "meta": {
        "profile_image_url": "/static/favicon.png",
        "description": "A model that is specialised to
assist
you in your daily coding work.",
        "capabilities": {
            "vision": false

```



```

        {
            "content": "Generate a REST API endpoint
in Node.js
using Express."
        },
        {
            "content": "How do I optimize a SQL query
for faster
performance?"
        },
        {
            "content": "Write a Python script to parse
JSON
data."
        }
    ]
},
"updated_at": 1723984665,
"created_at": 1723984665
},
"preset": true,
"actions": []
}
]

```

The big advantages of the combination of Open WebUI and Ollama over the popular online services is that your data doesn't leave your system, and you have complete control over the entire environment and all aspects of the model and interaction. As you've seen from the model configuration ex-

ample, you can intervene very deeply in the system and configure it entirely according to your requirements.